

Perfect. Here's **Implementation Spec v1** for the Seia Club / Miami Members Hospitality Toast reporting stack.

The architecture is now locked as:

Toast Standard API + PeopleVine enrichment → Fabric Lakehouse (bronze/silver) → Fabric gold layer / Warehouse → Power BI

That direction is supported by the research: Toast Standard API is the transaction-detail source of record, the payload must be split into separate grains instead of left as one big JSON blob, payments must be modeled separately from orders, and Fabric Lakehouse is the better landing zone for nested Toast JSON before serving curated analytics downstream.

1) Target production architecture

Source systems

- **Toast Standard API** for transactional operations
- **PeopleVine** for club/member/account attribution
- Later: optionally Tripleseat / Stripe, but not in the first Toast production slice

Fabric layers

Bronze

Raw append-only landing, exactly as received.

Silver

Normalized Toast and PeopleVine entities at operational grain.

Gold

Business-facing reporting marts for Power BI.

Serving

Prefer:

- **Direct Lake over curated Delta gold tables** for speed and simplicity first
- optionally add a **Fabric Warehouse gold layer** later if you want tighter T-SQL-managed star schemas and governance

The research strongly supports Lakehouse-first landing because Toast data is nested/semi-structured and needs medallion-style normalization before reporting.

2) Fabric object layout

Use these exact object names so the project stays organized.

Workspace

WS_SEIA_DataPlatform

Lakehouse

LH_SEIA_ToastOps

Warehouse

WH_SEIA_GoldReporting

Pipelines

- PL_TOAST_BOOTSTRAP
- PL_TOAST_INCREMENTAL
- PL_TOAST_CLOSE_RECON
- PL_TOAST_WEEKLY_REPAIR
- PL_PEOPLEVINE_INCREMENTAL
- PL_DIM_REFRESH
- PL_GOLD_BUILD

Notebooks / transformations

- NB_TOAST_AUTH_TEST
- NB_TOAST_ORDERS_RAW_TO_BRONZE
- NB_TOAST_PAYMENTS_RAW_TO_BRONZE
- NB_TOAST_NORMALIZE_ORDERS
- NB_TOAST_NORMALIZE_PAYMENTS
- NB_TOAST_NORMALIZE_LABOR

- NB_TOAST_NORMALIZE_CASH
- NB_PEOPLEVINE_NORMALIZE
- NB_BUILD_GOLD_MARTS
- NB_RECON_VALIDATIONS

Semantic model / Power BI

- SM_SEIA_OperationsFinance
- RPT_SEIA_Club_Executive
- RPT_SEIA_Finance_Recon
- RPT_SEIA_MemberSpend

3) Bronze layer design

Every bronze table should be append-only and include load metadata.

Common metadata columns

Include on every bronze table:

- ingested_at_utc
- source_system
- source_endpoint
- restaurant_guid
- record_source
Examples: ordersBulk, payments_api, orders_webhook, peoplevine_api
- extract_window_start_utc
- extract_window_end_utc
- payload_hash
- payload_json

Bronze tables

Toast

- brz_toast_orders_bulk
- brz_toast_orders_webhook
- brz_toast_payments
- brz_toast_restaurants
- brz_toast_menus
- brz_toast_menus_metadata
- brz_toast_config_revenue_centers
- brz_toast_config_sales_categories
- brz_toast_config_dining_options
- brz_toast_config_discounts
- brz_toast_config_service_charges
- brz_toast_config_tax_rates
- brz_toast_config_alt_payment_types
- brz_toast_config_tip_withholding
- brz_toast_config_cash_drawers
- brz_toast_config_no_sale_reasons
- brz_toast_config_payout_reasons
- brz_toast_employees
- brz_toast_time_entries
- brz_toast_cash_entries
- brz_toast_cash_deposits

PeopleVine

Since you already use PeopleVine successfully, I would formalize these raw feeds:

- brz_pv_members
- brz_pv_house_accounts
- brz_pv_invoices

- brz_pv_transactions
- brz_pv_subscriptions
- brz_pv_touchpoints
- brz_pv_member_accounts_bridge

For PeopleVine, the exact endpoint/report names can follow whatever exports/API calls you already trust today, but the warehouse should treat PeopleVine as the **member/account dimension system**, not as the core sales ledger.

4) Silver layer design

This is the heart of the implementation.

The research is very clear that Toast's hierarchical order model should be broken into separate tables: order, check, selection, modifier, discounts, service charges, payment, refund/void, and tax application layers.

Core Toast silver tables

stg_toast_order

Grain: one row per order

Key: restaurant_guid + order_guid

Fields:

- restaurant_guid
- order_guid
- business_date
- opened_date_utc
- closed_date_utc
- modified_date_utc
- promised_date_utc
- created_date_utc if available
- server_guid

- revenue_center_guid
- dining_option_guid
- source
- voided_flag
- deleted_flag
- ingested_at_utc
- record_source

stg_toast_check

Grain: one row per check

Key: restaurant_guid + check_guid

Fields:

- restaurant_guid
- check_guid
- order_guid
- business_date
- tab_name
- payment_status
- tax_exempt_flag
- amount_subtotal
- amount_tax
- amount_total
- opened_date_utc
- closed_date_utc
- modified_date_utc
- guest_reference
- ingested_at_utc

stg_toast_selection

Grain: one row per item selection

Key: restaurant_guid + selection_guid

Fields:

- restaurant_guid
- selection_guid
- check_guid
- order_guid
- business_date
- menu_item_guid
- menu_group_guid
- sales_category_guid
- quantity
- pre_discount_price
- price
- tax_amount
- voided_flag
- refund_flag
- fulfillment_status
- modified_date_utc

stg_toast_selection_modifier

Grain: one row per modifier line

Key: restaurant_guid + selection_guid + modifier_position

Fields:

- restaurant_guid
- parent_selection_guid
- modifier_position

- modifier_option_guid
- modifier_group_guid
- price
- tax_amount

stg_toast_payment

Grain: one row per payment

Key: restaurant_guid + payment_guid

Fields:

- restaurant_guid
- payment_guid
- order_guid
- check_guid
- paid_business_date
- void_business_date
- refund_business_date
- paid_date_utc
- amount
- tip_amount
- payment_type
- server_guid
- cash_drawer_guid
- card_payment_id
- tender_transaction_guid
- house_account_reference
- other_payment_reference
- modified_date_utc

- ingested_at_utc

This table is non-negotiable. The research confirms /payments is a separate required retrieval for finance-grade reporting and must not be inferred from orders alone.

stg_toast_payment_refund_void

Grain: one row per payment refund or void event

Key: restaurant_guid + payment_guid + event_type + event_ordinal

Fields:

- restaurant_guid
- payment_guid
- event_type
- event_amount
- event_business_date
- event_timestamp_utc
- void_reason
- refund_reason

stg_toast_check_service_charge

Grain: one row per applied service charge

Key: restaurant_guid + check_guid + service_charge_guid + ordinal

Fields:

- restaurant_guid
- check_guid
- order_guid
- service_charge_guid
- amount
- gratuity_flag
- taxable_flag
- destination

- business_date

stg_toast_check_discount

Grain: one row per check-level discount application

Key: restaurant_guid + check_guid + discount_guid + ordinal

stg_toast_selection_discount

Grain: one row per selection-level discount application

Key: restaurant_guid + selection_guid + discount_guid + ordinal

stg_toast_selection_tax

Grain: one row per selection tax application

Key: restaurant_guid + selection_guid + tax_guid + ordinal

Labor silver

- stg_toast_employee
- stg_toast_time_entry
- stg_toast_time_entry_break

Cash silver

- stg_toast_cash_entry
- stg_toast_cash_deposit

Dimension silver

- stg_dim_restaurant
- stg_dim_revenue_center
- stg_dim_sales_category
- stg_dim_dining_option
- stg_dim_discount
- stg_dim_service_charge
- stg_dim_tax_rate
- stg_dim_alt_payment_type
- stg_dim_cash_drawer

- stg_dim_no_sale_reason
 - stg_dim_payout_reason
 - stg_dim_menu_item
 - stg_dim_menu_group
 - stg_dim_modifier_group
 - stg_dim_modifier_option
-

5) PeopleVine enrichment layer

This is where the Seia-specific value really happens.

The research confirms Toast alone will not reliably solve club/member attribution, especially for dine-in and house accounts. PeopleVine needs to become the authoritative membership/account layer.

PeopleVine silver tables

I would create:

- stg_pv_member
- stg_pv_account
- stg_pv_house_account
- stg_pv_subscription
- stg_pv_invoice
- stg_pv_transaction
- stg_pv_touchpoint
- stg_pv_member_account_bridge

Key bridge tables

bridge_member_to_house_account

Maps:

- pv_member_id
- pv_account_id

- house_account_name
- house_account_email
- status
- effective_start_date
- effective_end_date

bridge_toast_payment_to_member

This is the critical Seia table.

Fields:

- restaurant_guid
- payment_guid
- check_guid
- order_guid
- house_account_reference
- toast_tab_name
- resolved_member_id
- resolved_account_id
- match_method
- match_confidence
- matched_at_utc

Matching logic

Use a priority stack:

1. direct Toast house-account reference ↔ PeopleVine house-account key
2. exact account/member name or tab-name mapping
3. invoice/transaction correlation by amount/date window
4. manually curated bridge overrides
5. unresolved bucket

Match method examples

- HOUSE_ACCOUNT_GUID_MATCH
- HOUSE_ACCOUNT_NAME_MATCH
- TAB_NAME_MATCH
- AMOUNT_DATE_INFERENCE
- MANUAL_OVERRIDE
- UNRESOLVED

This is what powers a real **member spend dashboard** instead of only a restaurant operations dashboard.

6) Gold model

Facts

- fact_sales_check
- fact_sales_selection
- fact_payments
- fact_discounts
- fact_service_charges
- fact_refunds_voids
- fact_labor_time_entry
- fact_cash_activity
- fact_member_spend

Dimensions

- dim_restaurant
- dim_date
- dim_business_date
- dim_employee

- dim_revenue_center
- dim_dining_option
- dim_menu_item
- dim_menu_group
- dim_sales_category
- dim_discount
- dim_service_charge
- dim_tax_rate
- dim_alt_payment_type
- dim_member
- dim_account

Marts for Power BI

Build these first:

- mart_daily_sales
- mart_daily_tenders
- mart_member_spend_daily
- mart_menu_mix_daily
- mart_discounts_comps_daily
- mart_service_charges_daily
- mart_voids_refunds_daily
- mart_labor_daily
- mart_cash_controls_daily
- mart_reconciliation_daily

7) Endpoint implementation order

The research already gave the right order. I would use it exactly.

Wave 1

- /restaurants/v1/restaurants/{restaurantGUID}
- /menus
- /metadata
- /config/v2/revenueCenters
- /config/v2/salesCategories
- /config/v2/diningOptions
- /config/v2/discounts
- /config/v2/serviceCharges
- /config/v2/taxRates
- /config/v2/alternatePaymentTypes
- /config/v2/tipWithholding

Wave 2

- /orders/v2/ordersBulk
- /orders/v2/payments

Wave 3

- /labor/v1/employees
- /labor/v1/timeEntries

Wave 4

- /cashmgmt/v1/entries
- /cashmgmt/v1/deposits
- /config/v2/cashDrawers
- /config/v2/noSaleReasons
- /config/v2/payoutReasons

That endpoint order is aligned to the research and should be your real build sequence.

8) Refresh and orchestration spec

Production orchestration tool

Use **Fabric Data Factory pipelines** as the orchestrator, with notebooks doing most normalization. The research explicitly supports Fabric pipelines for orchestration.

Bootstrap pipeline

PL_TOAST_BOOTSTRAP

Steps:

1. get restaurant metadata
2. get menus metadata
3. get menus
4. get config dimensions
5. backfill /ordersBulk in monthly windows
6. backfill /payments by business date
7. backfill employees/time entries/cash

Incremental pipeline

PL_TOAST_INCREMENTAL

Runs every 15–30 minutes:

1. refresh auth token only when needed
2. pull ordersBulk by modified window
3. land to bronze
4. normalize to silver
5. merge dimensions if changed
6. run lightweight DQ checks
7. update watermark

Close reconciliation pipeline

PL_TOAST_CLOSE_RECON

Runs at closeoutHour + 6h + buffer:

1. replay payments for current and prior 2 business dates
2. replay orders for overlap window
3. rebuild reconciliation marts
4. alert on variances

Weekly repair pipeline

PL_TOAST_WEEKLY_REPAIR

Runs weekly:

1. replay trailing 14 days orders
2. replay trailing 14 days payments
3. repair late changes
4. refresh unresolved member matches

PeopleVine pipeline

PL_PEOPLEVINE_INCREMENTAL

Runs daily or intraday depending on value:

1. refresh members/accounts/house accounts
2. refresh invoices/transactions
3. rebuild member bridges
4. reclass unresolved Toast payments/checks

9) Incremental rules

The research gives the core rules, and these should be treated as design requirements.

Orders

Use /ordersBulk with startDate/endDate on modified time, not just businessDate.

Recommended overlap: **30–60 minutes**.

Payments

Replay by:

- paidBusinessDate
- refundBusinessDate
- voidBusinessDate

Run nightly after closeoutHour + 6h.

Also replay current business date and prior 2 business dates on incrementals.

Time entries

Use modifiedStartDate/modifiedEndDate; do not treat active shifts as final.

Cash

Pull daily by businessDate.

Menus

Use /metadata as the stale check; reload full /menus only when needed.

Config

Daily refresh using lastModified where supported.

10) Control, lineage, and audit tables

Create these immediately.

- ctl_pipeline_run
- ctl_watermark
- ctl_endpoint_window_log
- ctl_rowcount_audit
- ctl_hash_change_audit
- ctl_recon_result
- ctl_unresolved_member_match
- ctl_manual_override_member_match
- ctl_data_quality_issue

ctl_watermark

Fields:

- source_system
- entity_name
- restaurant_guid
- watermark_type
- last_successful_start_utc
- last_successful_end_utc
- updated_at_utc

ctl_recon_result

Fields:

- business_date
- restaurant_guid
- metric_name
- source_value
- warehouse_value
- variance_value
- variance_pct
- status
- checked_at_utc

11) Reconciliation rules

You'll want two separate worlds in reporting:

Sales-side

Driven by:

- orders

- checks
- selections
- discounts
- service charges
- tax applications

Tender-side

Driven by:

- payments
- payment refunds
- payment voids

Do **not** collapse these into one fact.

Daily reconciliations

- checks total vs expected sales summary logic
- discounts total
- service charges total
- tax total
- payment amount total
- tip amount total
- refund amount total
- void amount total
- open checks count
- unmatched member-spend amount
- cash entry totals vs deposit totals

This split matters because the research confirms payments are separate from orders and business-date logic is not the same as event timestamps.

12) Security and environment requirements

Do these before real production work continues:

1. **Rotate the exposed Toast client secret immediately**
2. Store the new secret in a managed Fabric connection or approved secret store
3. Restrict workspace edit access
4. Confirm the credential includes:
 - orders:read
 - config:read
 - menus:read
 - restaurants:read
 - labor:read
 - labor.employees:read
 - cashmgmt:read
5. Document the Seia restaurant GUID and connection owner
6. Confirm the Fabric trial path to permanent supported capacity

The required scope set is explicitly supported in the research.

13) Webhooks plan

Do **not** make webhooks phase 1-critical.

Phase 1

Polling only:

- orders bulk replay
- payments replay
- config/menus checks
- labor/cash pulls

Phase 2

Add:

- Orders webhook → bronze
- Menus webhook → bronze trigger
- asynchronous processing after 2xx fast acknowledgment

That sequencing matches the research: orders webhook is useful, but not sufficient by itself for finance-grade completeness.

14) Implementation roadmap

Phase 1 — Foundation

- rotate secret
- create Fabric workspace objects
- build bronze landing
- implement restaurant + menu + config + orders + payments
- normalize order/check/selection/payment
- publish first finance recon model

Phase 2 — Operational completeness

- add labor
- add cash
- add full discount/service charge/tax breakdown
- publish operational dashboards

Phase 3 — Club enrichment

- load PeopleVine entities
- build member/account bridge
- build member spend mart
- build unresolved attribution workflow

Phase 4 — Hardening

- add webhooks
- add replay/repair automation
- add production alerts
- prepare multi-location expansion

That roadmap closely matches the research, with the main Seia-specific addition being formal PeopleVine enrichment.

15) First dashboards to build

Executive dashboard

- daily sales
- tenders collected
- comps/discounts
- refunds/voids
- top categories
- unresolved member spend

Finance dashboard

- sales vs tenders
- payment settlement timing
- refunds/voids trend
- service charges and tax
- business-date vs paid-date bridge

Club/member dashboard

- spend by member
- spend by account
- house-account aging
- visit frequency

- category mix by member
- unresolved or ambiguous attribution queue

Ops dashboard

- labor hours
 - labor-to-sales
 - cash controls
 - menu mix
 - dining option mix
-

16) Immediate next build tasks

This is the exact sequence I'd do next:

Next 5 tasks

1. rotate Toast secret
2. create the Lakehouse, Warehouse, and control tables
3. build bronze ingestion for:
 - restaurants
 - menus metadata
 - menus
 - config
 - ordersBulk
 - payments
4. build silver normalization for:
 - order
 - check
 - selection
 - selection modifier

- payment

5. build first reconciliation mart and first member bridge prototype from PeopleVine

17) Best next artifact to produce

The next thing that should exist is not another strategy memo. It should be one of these:

- **Fabric object-by-object build checklist**
- **table DDL / schema spec**
- **first notebook or M-code for ordersBulk**
- **PeopleVine bridge mapping spec**

The most valuable one is probably the **table DDL + merge-key spec**, because that turns this from architecture into something you can actually build against.